

4__Parameter__Server

March 28, 2022

1 ROS Parameter Server

Parameter Server jest zbiorem wartości parametrów odczytywanych przez nody w ROS. Parametry mogą przechowywać informacje o konfiguracji sprzętowej lub innych charakterystycznych parametrach. Serwer parametrów wykorzystuje 1 do 1 odpowiednik formatu YAML.

1.1 yaml przykład

string: 'jakis tekst'

integer: 3948

float: 231.5

boolean: false

list: [1.0, mixed list]

dictionary: {a: b, c: d}

Istnieje możliwość zmiany pojedynczych parametrów, ale można je też nadpisać słownikiem w rosparam. Oprócz samych wartości można wpisywać inne matematyczne operacje.

angle1: rad(3*pi)

angle2: !radians 3.14169

```
[8]: # Komendy dla polecenia rosparam
!rosparam --help
```

rosparam is a command-line tool for getting, setting, and deleting parameters from the ROS Parameter Server.

Commands:

rosparam set	set parameter
rosparam get	get parameter
rosparam load	load parameters from file
rosparam dump	dump parameters to file
rosparam delete	delete parameter
rosparam list	list parameter names

1.1.1 lista dostępnych parametrów

```
[9]: !rosparam list
```

```
/rostdistro  
/roslaunch/uris/host_localhost__38613  
/rosversion  
/run_id  
/turtlesim/background_b  
/turtlesim/background_g  
/turtlesim/background_r
```

1.1.2 odczyt pojedynczego parametru

```
[11]: # rosparam get cała_nazwa_parametru  
!rosparam get /turtlesim/background_b
```

```
255
```

```
[12]: # w przestrzeni nazw turtlesim znajdują się parametry:  
# background_b, background_g, background_r  
!rosparam get /turtlesim
```

```
background_b: 255  
background_g: 86  
background_r: 69
```

1.1.3 ustawianie nowego parametru

```
[13]: !rosparam set robot_1/tryb_pracy "auto"
```

```
[15]: # na liście parametrów pojawił się parametr /robot_1/tryb_pracy  
!rosparam list
```

```
/robot_1/tryb_pracy  
/rostdistro  
/roslaunch/uris/host_localhost__38613  
/rosversion  
/run_id  
/turtlesim/background_b  
/turtlesim/background_g  
/turtlesim/background_r
```

```
[16]: # wartość tego parametru wynosi
!rosparam get /robot_1/tryb_pracy
```

auto

```
[22]: # wykorzystanie słownika przy ustawianiu parametru
!rosparam set robot_2 "{v_max: 4, omega_max: 2}"
```

```
[24]: # na liście parametrów pojawił się parametry dla robot_2
!rosparam list
```

```
/robot_1/tryb_pracy
/robot_2/omega_max
/robot_2/v_max
/rosdistro
/roslaunch/uris/host_localhost__38613
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

1.1.4 usuwanie istniejącego parametru

```
[25]: !rosparam delete /robot_1/tryb_pracy
```

```
[27]: # z listy parametrów został usunięty parametr /robot_1/tryb_pracy
!rosparam list
```

```
/robot_2/omega_max
/robot_2/v_max
/rosdistro
/roslaunch/uris/host_localhost__38613
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

1.1.5 zapis aktualnych plików do .yaml

```
[26]: !rosparam dump dump.yaml
```

```
[31]: # zapis wybranej przestrzeni nazw do pliku
!rosparam dump -v bg_color.yaml /turtlesim
```

```
dumping namespace [/turtlesim] to file [bg_color.yaml]
/turtlesim/background_r=69
/turtlesim/background_g=86
/turtlesim/background_b=255
```

1.1.6 import parametrów z pliku .yaml

```
[28]: !rosparam load nowe_parametry.yaml
```

```
[29]: !rosparam list
```

```
/robot_2/omega_max
/robot_2/v_max
/robot_3/goal_tolerance
/robot_3/omega_max
/robot_3/tryb
/robot_3/v_max
/robot_4/goal_tolerance/angle
/robot_4/goal_tolerance/x
/robot_4/goal_tolerance/y
/robot_4/omega_max
/robot_4/tryb
/robot_4/v_max
/rostdistro
/roslaunch/uris/host_localhost__38613
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

```
[32]: # import parametrów z pliku pod daną przestrzeń nazw np. kolor_tla
!rosparam load bg_color.yaml /kolor_tla
```

```
[33]: !rosparam list
```

```
/kolor_tla/background_b
/kolor_tla/background_g
/kolor_tla/background_r
/robot_2/omega_max
/robot_2/v_max
/robot_3/goal_tolerance
/robot_3/omega_max
/robot_3/tryb
/robot_3/v_max
/robot_4/goal_tolerance/angle
/robot_4/goal_tolerance/x
```

```
/robot_4/goal_tolerance/y
/robot_4/omega_max
/robot_4/tryb
/robot_4/v_max
/rosdistro
/roslaunch/uris/host_localhost__38613
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

2 Obsługa w języku Python

2.1 Lista dostępnych parametrów

```
[34]: import rospy
      rospy.get_param_names()
```

```
[34]: ['/run_id',
      '/roslaunch/uris/host_localhost__38613',
      '/rosversion',
      '/rosdistro',
      '/turtlesim/background_r',
      '/turtlesim/background_g',
      '/turtlesim/background_b',
      '/robot_2/v_max',
      '/robot_2/omega_max',
      '/robot_3/omega_max',
      '/robot_3/v_max',
      '/robot_3/goal_tolerance',
      '/robot_3/tryb',
      '/robot_4/omega_max',
      '/robot_4/v_max',
      '/robot_4/goal_tolerance/x',
      '/robot_4/goal_tolerance/y',
      '/robot_4/goal_tolerance/angle',
      '/robot_4/tryb',
      '/kolor_tla/background_b',
      '/kolor_tla/background_g',
      '/kolor_tla/background_r']
```

2.2 Odczyt wartości parametru

```
[35]: print(rospy.get_param("turtlesim/background_r"))
      print(rospy.get_param("turtlesim/background_g"))
      print(rospy.get_param("turtlesim/background_b"))
```

```
69
86
255
```

Ustawienie nowych wartości parametrów tła symulacji turtlesim Aby wprowadzone zmiany były widoczne należy wywołać dodatkowo serwis `/clear`

```
[36]: rospy.set_param("turtlesim/background_r", 255)
      rospy.set_param("turtlesim/background_g", 0)
      rospy.set_param("turtlesim/background_b", 0)
```

```
[37]: # dla symulacji z turtlesim wywołanie serwisu clear po zmianie parametrów tła
      ↪ powoduje zmianę
      # koloru tła
      !rosservice call /clear
```

```
[44]: # surowe obiekty w Pythonie
      rospy.set_param('robot_on', False)
      # przekazanie wartości jako słownik
      rospy.set_param('gains', {'p': 1, 'i': 2, 'd': 3})
```

```
[45]: # biblioteka do obsługi serwera parametrów (alternatywne podejście zamiast
      ↪ rospy.set_param())
      import rosparam
      # yaml string jako wartość
      rosparam.set_param('robot_on', "true")
      rosparam.set_param('gains', '{"p": 1, "i": 2, "d": 3}')
```

```
[47]: !rosparam get /robot_on
```

```
true
```

Ustawianie kilku różnych parametrów w określonej przestrzeni nazw. Na przykładzie kolorów RGB tła można by ustawić je w przestrzeni nazw **color** i zgrupować razem. Takie ustawienie parametrów nie będzie miało wpływu na środowisko symulacyjne (turtlesim), gdyż nie są one obsługiwane.

```
[62]: grupa = "color"
      rospy.set_param(grupa+"/r", 41)
      rospy.set_param(grupa+"/g", 231)
      rospy.set_param(grupa+"/b", 0)
```

```
[50]: rospy.get_param("color")
```

```
[50]: {'r': 41, 'g': 231, 'b': 0}
```

2.3 sprawdzanie czy parametr istnieje

```
[52]: rospy.has_param('robot_on')
```

```
[52]: True
```

```
[53]: rospy.has_param('ten_parametr_nie_istnieje')
```

```
[53]: False
```

2.4 Usuwanie parametrów

Do usunięcia pojedynczego parametru należy podać jego pełną nazwę.

```
[63]: rospy.delete_param("/color/r")
```

Można usunąć więcej parametrów znajdującej się w określonej przestrzeni nazw.

```
[64]: rospy.delete_param("/color/")
```

W trakcie próby usunięcia nieistniejącego parametru pojawia się błąd

```
[65]: rospy.delete_param("/color/")
```

```
↳ -----  
KeyError                                Traceback (most recent call last)  
  
  /tmp/ipykernel_995/569920000.py in <module>  
----> 1 rospy.delete_param("/color/")  
  
  /opt/ros/noetic/lib/python3/dist-packages/rospy/client.py in  
↳ delete_param(param_name)  
    566     """  
    567     _init_param_server()  
--> 568     del _param_server[param_name] #MasterProxy does all the magic for  
↳ us  
    569  
    570 def has_param(param_name):
```

```

/opt/ros/noetic/lib/python3/dist-packages/rospy/msproxy.py in
↳ __delitem__(self, key)
    184         code, msg, _ = self.target.deleteParam(rospy.names.
↳ get_caller_id(), resolved_key)
    185         if code == -1:
--> 186             raise KeyError(key)
    187         elif code != 1:
    188             raise rospy.exceptions.ROSException("cannot delete
↳ parameter: %s"%msg)

```

```

KeyError: '/color/'

```

```

[66]: # Pojawił się błąd KeyError, aby uniknąć nagłego przerwania działania programu
↳ można obsłużyć
# wyjątek z wykorzystaniem składni try except. Błąd typu KeyError, który pojawia
↳ się w try
# jest obsługiwany w części except i pojawia się informacja "value not set". Nie
↳ pojawia się
# już błąd tak jak w powyższym przykładzie.
try:
    rospy.delete_param('to_delete')
except KeyError:
    print("value not set")

```

```

value not set

```

2.5 wyszukiwanie parametrów po nazwie klucz

```

[99]: param_name = rospy.search_param('robot_1')
v = rospy.get_param(param_name)

```

```

[100]: param_name

```

```

[100]: '/robot_1'

```

```

[101]: v

```

```

[101]: {}

```

```

[ ]:

```